Check for updates

Article

Journal of Educational Computing Research
2024, Vol. 62(5) 1165–1195
© The Author(s) 2024
Article reuse guidelines:
sagepub.com/journals-permissions
DOI: 10.1177/07356331241240460
journals.sagepub.com/home/jec
Sage

Toward Artificial Intelligence-Human Paired Programming: A Review of the Educational Applications and Research on Artificial Intelligence Code-Generation Tools

Jiangyue Liu[1] and Siran Li[1]

Abstract

Pair Programming is considered an effective approach to programming education, but the synchronous collaboration of two programmers involves complex coordination, making this method difficult to be widely adopted in educational settings. Artificial Intelligence (AI) code-generation tools have outstanding capabilities in program generation and natural language understanding, creating conducive conditions for pairing with humans in programming. Now some more mature tools are gradually being implemented. This review summarizes the current status of educational applications and research on AI-assisted programming technology. Through thematic coding of literature, existing research focuses on five aspects: underlying technology and tool introduction, performance evaluation, the potential impacts and coping strategies, exploration of behavioral patterns in technological application, and ethical and safety issues. A systematic analysis of current literature provides the following insights for future academic research related to the practice of "human-machine pairing" in programming: (1) Affirming the value of AI code-generation tools while also clearly defining their technical limitations and ethical risks; (2) Developing adaptive teaching

[1]Soochow University, Suzhou, China

Corresponding Author:
Siran Li, School of Education, Soochow University, 199 Ren'ai Road, Suzhou Industrial Park, Suzhou, Jiangsu, China.
Email: 20224218008@stu.suda.edu.cn

ecosystems and educational models, conducting comprehensive empirical research to explore the efficiency mechanisms of AI-human paired programming; (3) Further enriching the application of research methods by integrating speculative research with empirical research, combining traditional methods with emerging technologies.

Keywords
pair programming, code-generation tools, educational applications, underlying technology, ethical issues

Introduction

In higher education settings, programming remains a challenging area for novice programmers. Previous studies indicate that beginners face difficulties in conceptualizing programming problems, designing solution strategies, and executing and evaluating solutions due to the abstract nature and complexity of programming learning content (Medeiros et al., 2019). This challenge is particularly evident in introductory programming courses at the university level, with global statistics showing that approximately one-third of students struggle to pass, leading to high dropout and failure rates (Bennedsen & Caspersen, 2007; Watson & Li, 2014). Furthermore, in real-world professional scenarios, given the complexity of modern programs or systems, one person cannot complete all development tasks. Tasks involve coordination and collaboration across multiple departments at various stages, turning programming into a collaborative intellectual task. Therefore, fostering students' collaborative skills is crucial in programming courses. To alleviate the common learning difficulties in novice programming and meet the future talent needs of the workforce, collaborative programming is widely practiced in higher education.

"Pair Programming" (PP) is a collaborative programming approach where two programmers work together on the same programming task. One assumes the role of the "Driver," controlling the keyboard and mouse and writing code, while the other serves as the "Navigator," consistently observing, monitoring program logic, and providing auxiliary support. Whenever issues arise, both parties discuss and solve them jointly (Xu et al., 2023). Empirical studies indicate that the application of PP in educational settings can stimulate the learning motivation and interest of novice programmers in computer science, cultivate their computational thinking abilities, and enhance their problem-solving skills (Beck & Chizhik, 2013; Chittum et al., 2017; Romero et al., 2017). However, PP is a complex phenomenon involving intricate interactions and coordination between individual students, student groups, the learning environment, and knowledge products (Stahl & Hakkarainen, 2021). As PP is an interpersonal collaborative process, pairing compatibility might be influenced by human-related factors such as personalities (Sfetsos et al., 2006). In this process, issues like free-riding and unequal levels of cooperation persist (Häkkinen et al., 2010).

Moreover, for individuals, pair programming may potentially reduce their programming efficiency (Müller, 2004). Consequently, due to the complexity involved in effectively coordinating two programmers to work synchronously, widespread adoption of this method in educational settings remains challenging.

The emergence of Artificial Intelligence (AI) code-generation tools has introduced a new form of "AI-human pairing" for PP. On one hand, in the field of software engineering, programmers often encounter a substantial volume of repetitive and technically low-complexity coding tasks. On the other hand, AI, compared to humans, tends to be more "rational," potentially addressing compatibility issues when paired with humans. However, in the collaboration between humans and AI, the AI does not bear responsibility for the final learning outcomes. Therefore, there is a greater need for humans to act as the "Navigators" in the collaborative process. This involves strategically considering the direction of tasks, autonomously questioning, analyzing, and critically evaluating the content generated by AI, and actively engaging in knowledge construction aimed at problem-solving, which is more conducive to unleashing students' "cognitive agency" (Rudolph, 2023) and fostering critical thinking development (Eysenbach, 2023). Additionally, since the release of OpenAI's ChatGPT 3.5 on November 30, 2022, large language models (LLMs) and Natural Language Processing (NLP) technologies have gained significant attention. Generative Artificial Intelligence (GAI), such as ChatGPT, has rapidly garnered worldwide interest. GAI's efficient natural language understanding and generative capabilities show immense potential to revolutionize the traditional patterns of PP. The integration of natural language processing with code-generation tools creates ample conditions for facilitating the collaborative "AI-human paired" process in programming. Consequently, there is an increasing need to comprehend the value and significance of using such tools in education.

Currently, there is no review in the academic community regarding the educational applications and research status of AI code-generation tools. This study aims to address this gap. We pose the following three research questions:

(1) What topics are currently being focused on in the educational application and related research of AI code-generation tools?
(2) What trends and characteristics are generally observed in the overall research in this field?
(3) What aspects should future research focus on?

This article is further divided into the following sections. We begin by reviewing the development history of "AI Code-Generation Tools" and further define its concept. The "Methodology" section discusses the approach taken in this research by showcasing the key steps involved in reviewing relevant studies. Subsequently, the article introduces results and findings by elucidating answers to research questions derived from a comprehensive review of the selected studies. Finally, the "Conclusion and Discussion"

section summarizes the study, emphasizing the main discoveries and achievements of this work.

Development of AI Code-Generation Tools

Empowering machines with the capability to comprehend user requirements and achieve semi-automated or automated program writing has been a longstanding emphasis in the domain of software engineering. To achieve this objective, researchers initially attempted to utilize historical information (Robbes & Lanza, 2010) and coding patterns extracted from software repositories (Hindle et al., 2016) to address simple programming problems. As technological advancements unfolded, the introduction of Deep Learning (DL) models became prominent, exemplified by White et al. (2015) who proposed DL models specifically tailored for source code files and subsequently applied them to enhance code suggestions.

Subsequent research in the domain of programming automation has expanded beyond mere code completion and suggestions, shifting its focus toward the generation of code blocks based on natural language comprehension. Researchers explored state-of-the-art Transformer-based models' capabilities in supporting various granularity levels of code completion, ranging from individual words, and one or more complete sentences, to entire code blocks, presenting viable solutions for code auto-generation (Mastropaolo et al., 2023). The release of GitHub Copilot further accelerated the realization of the dream of programming automation. Copilot, developed by OpenAI, leverages OpenAI's Codex model (built on top of OpenAI's language generation model GPT-3) and its extensive training to automatically generate complete code from human comments and natural language descriptions of program functionalities. Alongside GitHub Copilot, tools like DeepMind AlphaCode and Amazon CodeWhisperer have also garnered substantial attention for their applications in AI-assisted programming. Since the release of OpenAI's ChatGPT 3.5 on November 30, 2022, discussions about GAI have surged in society. GAI, built upon large pre-trained language models and trained on vast amounts of unlabeled data, exhibits outstanding code generation performance. The integration of potent AI capabilities into Integrated Development Environments (IDEs) like Visual Studio Code, Microsoft Visual Studio, Vim, Cursor, or JetBrains represents an imminent future trend. Consequently, the collaborative partnership between humans and AI in programming is poised to become increasingly prevalent.

In this review, "AI code-generation tools" refer to the tools that are built upon Large Language Models (LLM) and integrate natural language processing capabilities. Given that these relatively advanced tools were officially introduced and progressively integrated into extensive usage post-2022, this study confines its search parameters, commencing from the year 2022.

Methodology

This article aims to conduct a review of the present state of technical applications and educational research in AI-assisted programming tools within the education domain. It adheres to the PRISMA principles (Liberati et al., 2009) for article selection, and the complete process is illustrated in Figure 1.

Search

The study utilized the Web of Science and the Springer Link Digital Library as literature sample databases, supplemented by Google Scholar to capture potentially overlooked publications. Employing a Boolean expression ((‘Computing’ OR ’Programming’ OR

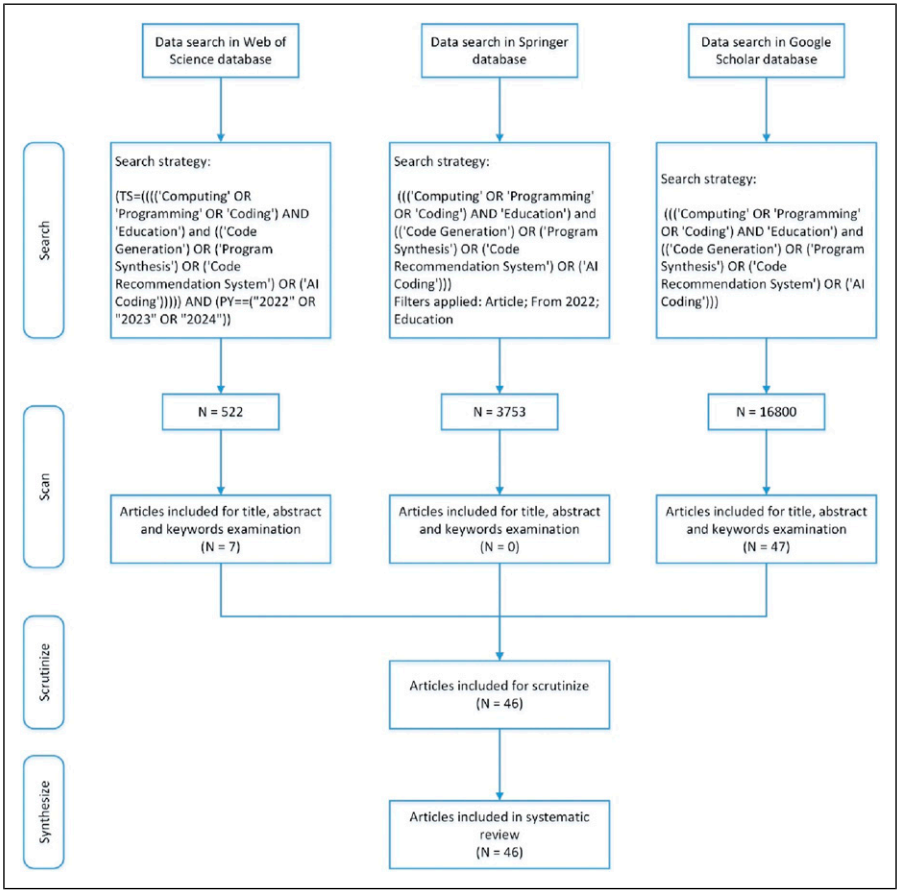

Figure 1. Article selection process.

'Coding') AND 'Education') and (('Code Generation') OR ('Program Synthesis') OR ('Code Recommendation System') OR ('AI Coding')) to match the themes of published journals and conference papers, within the field of "Education" in WoS and Springer databases. The research set the starting search date as 2022 and the end date as November 27, 2023. Upon preliminary searches, WoS and Springer respectively yielded 522 and 3753 results, while Google Scholar returned 16,800 results.

Scan

At this stage, each author meticulously reviewed the titles, abstracts, and keywords of the literature, conducting initial screening and filtering based on the inclusion and exclusion criteria outlined in Table 1. This process involved two meetings to discuss any discrepancies in the selection criteria and to reach a consensus on whether to include or exclude specific literature. As a result, 54 articles were retained. Most of the literature excluded from Google Scholar was unrelated to education, while the excluded literature from WoS and Springer mostly did not pertain to AI code-generation tools.

Scrutinize

At this phase, researchers conducted a more refined screening of the literature based on the established inclusion and exclusion criteria. Simultaneously, duplicate literature retrieved from different databases was identified and removed. Additionally, a comprehensive content analysis of the selected articles was conducted, focusing on the research objectives identified, the AI-assisted programming tools cited, the research methodology employed, and the resulting research conclusions. Table 2 outlines the coding scheme for the literature content analysis. Following this scheme, two researchers independently coded all articles using a consistent coding protocol, resulting in an agreement of 84% between the coders. Any discrepancies in coding were meticulously discussed and resolved to ensure accuracy and consistency. Following this

Table 1. Inclusion and Exclusion Criteria.

Criteria	Inclusion criteria	Exclusion criteria
Language	English	Not English
Type	Publicly available or archived journal articles, conference proceedings, or preprints	Introductions to books and/or seminars, book chapters, dissertations
Content	Taking AI-assisted programming tools as the main research content	Does not involve AI-assisted programming tools
	AI used in education	The target application is not an educational or learning technology

Table 2. Document Content Coding System.

Coding dimensions	Coding items
Research purposes	-
Mentioned AI code-generation tools	-
Participants	Students, teachers, experts, others, mixed or no participants
Methods	Quantitative methods, qualitative methods, or mixed methods
Conclusions	-

rigorous phase of literature scrutinization and in-depth content analysis, a final set of 46 pertinent articles was attained.

Synthesize

Through the preliminary content-coding of literature, researchers employed qualitative coding to identify and synthesize research themes in the field of AI code-generation tools (as shown in Table 3). Some of the literature covers two or more research themes.

Results

The Current Primary Research Themes within the Field

According to the analysis of the literature finally included in the study, the existing research focuses on five elements relating to AI code-generation tools: underlying technology and tool introduction, performance evaluation, the potential impacts and coping strategies of educational applications, the exploration of behavioral patterns in technical application, and ethical and security issues.

Underlying Technology and Tool Introduction. In total, 23 articles focus on the underlying technology and introduction of AI code-generation tools. Prior to the extensive deployment of popular AI code-generation tools like Copilot, code generation has attracted interest from the academic and industrial realms as support for developers in terms of programming. This prompts many studies on the technical aspects of AI-based code-generation tools. In some research, the current approaches to DL-based code generation are categorized into three types: those reliant on code features, those incorporating various retrieval methods, and those integrating post-processing techniques (Yang et al., 2023). In those of the first type, the code features in code generation are used through deep learning algorithms. Those of the other types are developed through the improvement made on the first type.

Table 3. Literature Topic Analysis Results.

Research topics	Quantity	Main research content	Research methodology
Underlying technology and tool introduction	23	• Underlying technical implementation of AI code-generation tools • Features of mainstream code-generation tools	• Discursive research • Comparative research
Performance evaluation of AI code-generation tools	25	• Performance evaluation study of AI code-generation tools • Study of the experience of using AI code-generation tools	• Contrasting experimental research • Discursive research • Quasi-experimental research
The potential impacts and coping strategies of educational applications	28	• Application and potential impact of AI code-generation technology in teaching and learning • Response to programming education and talent cultivation under the impact of technology	• Discursive research • Educational science fiction stories • Quasi-experimental research
Exploration of behavioral patterns in technological application	5	• Exploration of programming behavioral processes of AI-human pairing	• Rooted theory research • Behavioral coding based on structured observation methods • Discursive research
Ethical and safety issues	12	• Ethicality and safety issues in the application of AI code-generation tools	• Case studies • Quasi-experimental studies

Regarding the first type, the task of code generation is treated as a problem with machine translation, with natural language descriptions transformed into abstract code. The techniques of machine translation are based on mainstream sequence-to-sequence models. During the training phase, these models are assigned extensive sets of natural text-code pairs (<Text, Code>) to learn the interconnectedness between natural language and bimodal data, including code. Moreover, in the field of code generation, large-scale pre-trained Transformer models have been integrated to improve model performance by incorporating the existing methodologies of code feature extraction into pre-training paradigms. At present, the typical examples in the first category include GitHub Copilot, DeepMind AlphaCode, and the CodeGeeX large-scale code generation pre-trained model jointly developed in the Peng Cheng Laboratory at Tsinghua University. For instance, GitHub Copilot is empowered by the new AI system of OpenAI, namely OpenAI Codex, which has been trained on billions of lines of publicly available code. It facilitates code completion, suggestions, and comment-based code auto-generation, as shown in Figure 2.

This study aims to thoroughly examine the functionalities of well-developed AI-assisted programming platforms and analogous tools based on a comprehensive analysis and review of the relevant literature, as shown in Table 4 and Table 5.

In some studies, the current mainstream code-generation tools are introduced. Zheng et al. (2023) presented a wide-ranging overview of the architecture and training performance of CodeGeeX, demonstrating its effectiveness in enhancing the efficiency of coding. Becker et al. (2023) proposed a number of feasible and freely accessible tools for AI-driven code generation, analyzing the potential opportunities and risks associated with AI code-generation.

Performance Evaluation of AI Code-generation Tools. In total, 25 articles focus on evaluating the performance of AI code-generation tools. There have been some studies conducted on the performance of AI-assisted programming tools, a majority of which revolve around the most used tools of code generation: Github Copilot, Codex, and ChatGPT.

A total of 8 articles focus on comparatively evaluating the performance of AI program generation tools. Depending on different research objectives, these studies can be categorized into four types: "comparison of different tools," "human-human comparison," "human-AI comparison," and "comparison of performance in solving different types of tasks." In terms of research conclusions, these studies center on the speed and accuracy of completing tasks via AI code-generation tools.

Sobania et al. (2023) drew a comparison in error-fixing performance between ChatGPT, CoCoNut, Codex, and Standard APR. According to the study, ChatGPT resolved 19 out of 40 errors, performing as well as CoCoNut (19) and Codex (21), and better than standard APR (7). When researchers provided the specific prompts related to input-output content to ChatGPT, its error-fixing performance was significantly improved, with a total of 31 errors out of 40 fixed.

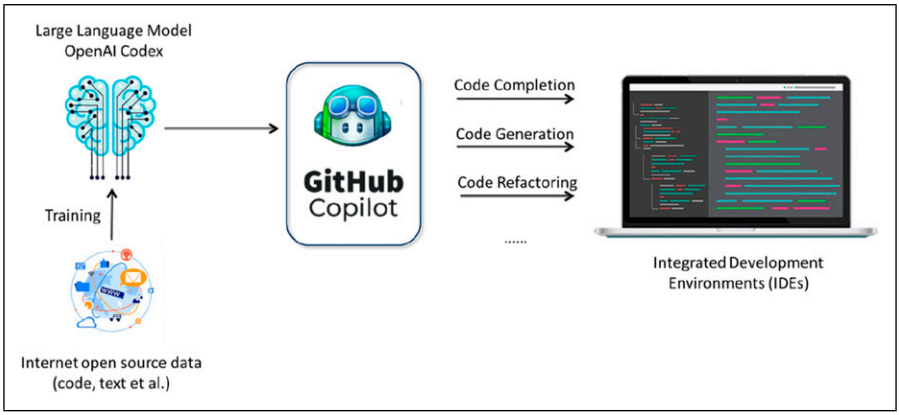

Figure 2. Feature implementation of GitHub copilot.

Peng et al. (2023) recruited 95 software developers, of whom 45 were in the experimental group and the remaining 50 were in the control group. Then, they were tasked with the rapid implementation of an HTTP server via JavaScript. The programmers in the control group, who completed programming tasks independently, took an average time of 71.17 minutes to complete them. In comparison with the control group (p-value = .0017), the experimental group, in which AI-assisted pair programming was conducted, took a 55.8% faster completion time (160.89 minutes), indicating a significant improvement by AI in the efficiency of programming. Additionally, the success rate of task completion in the experimental group was 7% higher than the control group, despite no statistical significance of this difference.

By assessing the performance of Github Codex in completing introductory programming exam questions, Finnie-Ansley et al. (2022) compared it with the scores of students who took the same exams as normal. Codex was used to complete two sets of test questions, which involved a total of 23 programming problems. Ultimately, Codex scored 15.7/20 (78.5%) on Test 1 and 19.5/25 (78.0%) on Test 2. By averaging the scores from both tests, it was found out that Codex ranked 17th among 71 students, making it in the top quarter of the class. This indicates that Codex outperformed most students in solving CS1 programming tasks. In a subsequent study, Finnie-Ansley et al. (2023) evaluated the performance of Codex in solving more advanced CS2 (data structures and algorithms) exam questions. According to the results, Codex ranked 66th among 264 students, making it in the top quarter of the class. Codex was more accurate in solving more complex programming problems than most students.

Dakhel et al. (2023) compared the performance of Copilot with student programmers. The results show that the average correctness rate (CR) of human solutions (CR = 0.59) is higher than Copilot (CR@Top1 = 0.52, CR@Top5 = 0.43, CR@Top10 = 0.35). In this context, TopK represents the top K solutions returned by Copilot, CR@

Table 4. Current Mainstream AI-Assisted Programming Platforms.

Categories		GitHub copilot	Amazon CodeWhisperer	DeepMind AlphaCode	CodeGeeX
Developer		Github OpenAI	Amazon	Google DeepMind	Tsinghua university, China
Release date		Preview release in Jun 2021; official release and open access in Jun 2022	Preview release in Jun 2022, official release and open access in Apr 2023	Released in Feb 2022	Sep 2022 open source plug-in free and open access
Supported languages	Python	√	√	√	√
	JavaScript	√	√		√
	TypeScript	√			
	Ruby	√			
	Go	√			√
	C#	√			
	C++	√		√	√
	Java	√	√		√
Supported code editors	VS code	√	√		√
	JetBrains IDE	√	√		√
	Visual studio	√			
	Neovim	√			
	AWS Cloud9		√		
	AWS lambda		√		

Top1 denotes the average correctness rate of the first solution returned by Copilot, CR@Top5 indicates the average correctness rate of the top 5 solutions returned by Copilot, and CR@Top10 refers to the average correctness rate of the top 10 solutions returned by Copilot. However, the average bug-fixing rate of Copilot for incorrect solutions generated is 0.95, suggesting that 95% of the bugs generated by Copilot are fixed, exceeding the bug-fixing rate of 0.89 as achieved by students. This suggests that the incorrect solutions generated by Copilot require less effort for rectification.

Additionally, Dakhel et al. (2023) conducted a comparative analysis of Copilot's performance in addressing various programming problems. When tasked with implementing simple sorting algorithms such as bubble sort, bucket sort, and insertion sort, Copilot provided highly accurate code in the first attempt (with correctness rates of 100%, 85.71%, and 100%, respectively). However, implementing heap sort, which

Table 5. Summary of the Functions of AI-Assisted Programming Tools.

Type	Function description	Explanation
Code generation	Code completion	Automatically perform code completion based on the context of the user's programming process
	Code generation	Generate statements, code blocks, or functions based on the user's natural language descriptions. For example: "Create a function that takes a list of first names and a list of last names and then returns a new list with those names."
	Code refactoring	Request tools to rewrite completed code for improved readability or maintainability. For instance, "refactor this function to use smaller helper functions."
	Language translation	Facilitate mutual conversion between different programming languages
	Test generation	Enable users to request AI programming tools to generate test cases (e.g., unit/regression tests). For example, Copilot's TestPilot feature (Schäfer et al., 2024) can generate tests and interactively refine them based on user feedback. These tools can often create tests for unusual edge cases that novices might not consider themselves (Simon, 2022)
Code explanation	Code explanation	An AI programming tool can take a code snippet and natural language instructions as input and explain the purpose of the code in a manner akin to how a human instructor explains to a student
	Debugging help	Users can request the tool to identify potential bugs in the provided code and explain why these could be bugs (Markel & Guo, 2021)

involves creating a max heap and then writing a sorting function, proved to be more challenging. When asked to implement heap sort, Copilot yielded a correctness rate of 0% in the first attempt and 9.09% in the second attempt, indicating suboptimal performance. In comparison to sorting algorithms, the implementation of greedy algorithms is more intricate. When researchers instructed Copilot to address four sub-problems related to designing a greedy algorithm, the average correctness rate of the initially generated code was 5.71%, and after a second attempt, the average correctness rate for the four sub-problems decreased to 4.17%. It is evident that as algorithm complexity increases, Copilot's problem-solving performance declines.

Megahed et al. (2023) compared the performance of ChatGPT given various task resolutions. By evaluating the code generated by ChatGPT, it was found out that ChatGPT performed well in structured tasks, such as translating the code from one language to another and explaining those widely known concepts. However, its performance declined when more intricate complete programming tasks or ill-

structured problems were dealt with, such as the explanation of less commonly known terms or the generation of code from scratch.

In the remaining literature, the data on problem-solving performance was collected during the use of certain AI code-generation tools, and their performance was evaluated using both quantitative and qualitative methods.

In some studies, quantitative data on the problem-solving performance of the tools was collected. For instance, Denny et al. (2023) evaluated the problem-solving performance of Copilot on a publicly available dataset of 166 programming problems. As revealed by the research, Copilot resolved approximately half of the problems on the first attempt. Moreover, it resolved 60% of the remaining problems only by further enhancing or refining the description of problems through natural language.

There is some literature focusing on the user experience brought by AI code-generation tools, primarily through an investigation into the subjective experiences of students and the value perceived by them during the usage of such tools through various qualitative research methods such as interviews. These studies center on the satisfaction of users with the tools and their trust in them. Prather et al. (2023) conducted structured interviews and observed introductory-level students using Github Copilot to complete introductory programming (CS1) assignments. They explored the perceptions of students about the advantages and disadvantages of this technology, summarizing the cognitive and metacognitive difficulties encountered by students. According to the research, beginners found it challenging to comprehend the content generated by Copilot, taking caution towards the practical application of these tools. However, they remain optimistic about the use of such tools in their future development work. Vaithilingam et al. (2022) conducted a user study on 24 participants to understand how programmers use and perceive Copilot. As revealed by the study, although Copilot might not necessarily shorten task completion time or improve the success rate, most of the participants preferred using Copilot in their daily programming tasks because of the reduced effort made on online searches.

In general terms, the existing research underscores the considerable potential of code-generation tools integrated with the capability of natural language processing to enhance task-solving performance significantly. However, there is still no unified conclusion reached from the comparison in accuracy between AI code-generation tools and human programming. As the overall trend suggests, AI outperforms most students in educational contexts and can be applied to complete programming tasks faster. However, it's noteworthy that AI is effective in completing simple programming tasks or answering well-structured questions, but its performance may deteriorate in the case of complex problems or comprehensive programming tasks. In terms of research methodology, empirical research methods are mainly adopted to evaluate the performance of AI code-generation tools, especially comparative studies, interviews, and other survey-based methods.

The Potential Impacts and Coping Strategies of Educational Applications. Focusing on the application of AI code-generation technology in education, a total of 28 papers analyze

and propose the corresponding strategies. These include speculative research, survey studies, and science fiction narratives outlining the blueprint for future programming education. Also, based on empirical research, careful consideration is given to whether AI code-generation tools can be incorporated into educational practices.

Regarding the potential impact of applying AI code-generation tools in education, these studies highlight their significance to enhancing teaching and supporting the learning by students. In the field of computer education, researchers emphasize that AI code-generation tools can generate solutions for programming assignments and examination queries (Denny et al., 2023; Finnie-Ansley et al., 2022, 2023; Wermelinger, 2023). Moreover, these tools can provide insightful explanations for code content (Denny et al., 2022; MacNeil et al., 2022; Sarsa et al., 2022), contributing significantly to educators' efficiency while fostering personalized and independent learning for students.

Meanwhile, these tools promote the cultivation of higher-order thinking ability for students. As argued by Denny et al. (2023), the process of interaction between students and AI, where students direct the generation of correct code through natural language, represents a potentially valuable learning activity. This activity has the potential to improve the computational thinking skills of students.

Additionally, the application of AI code-generation tools has the potential to reduce the barriers to entry for introductory programming. During a thorough analysis of 20 science fiction movies, Krings et al. (2023) identified and investigated some new approaches to programming as depicted in these films, including programming through bodily interaction, programming through natural language, and programming through music or acoustic signals. Furthermore, it was observed that the current science fiction movies are common in envisioning a future where everyone can do programming. Afterward, graduates were prompted by researchers to envision the potential impact of the popularization of software development on business and society by writing science fiction stories. Under these speculative scenarios presented in social science fiction, it was inferred that a society where "everyone can code" might cause various consequences, including the rising demand for programming education in schools, attached more significance to the communication and thought processes among individuals, and the potential market shifts affecting the roles of programmers. Based on speculative science fiction narratives, the insights gained from this research have meaningful implications for contemporary education and society alike.

However, there remain some scholars skeptical of the value created by applying such tools in teaching. Jayagopal et al. (2022) adopted AI code-generation tools to conduct teaching practices and performed a quasi-experimental study to reveal that the students applying AI code-generation tools for programming learning and task completion in the experimental group performed tasks faster than those who did independent programming in the control group. However, there was no evidence of any significant variation in learning outcomes between the two groups, which implies that improving the speed of programming alone is not necessarily a sufficient condition for the popularization of such tools in educational practices.

In summary, the current research has given recognition to the potential impact of AI code generation on the present practices of programming learning and the future patterns of the labor market. Therefore, the corresponding solutions and strategies are increasingly explored. Researchers support the changes in programming education related to various teaching objectives and teaching processes.

Concerning the objectives of programming education, it is suggested by the current researchers that its focus has shifted from nurturing code-writing proficiency to emphasizing code assessment and comprehension. Finnie-Ansley et al. (2022) suggest laying more emphasis on the ability of students to assess code rather than on their ability to write code. Likewise, Bird et al. (2022) pointed out that the role of developers can change when AI-driven tools are integrated into more tasks of software development. Therefore, more time should be spent on evaluating the task-related suggestions made by AI rather than on completing the tasks. In the view of Wermelinger (2023), AI-driven programming tools allow students to spend less time typing code and understanding compiler errors. During teaching, instructors and teaching assistants can focus more on program testing, debugging, and program comprehension as a result.

Regarding programming education, there is a significant risk of academic dishonesty posed as the influence of AI program generation tools grows. In a recent study focusing on the impact of OpenAI Codex on computer science education, Lau and Guo (2023) conducted interviews with some university computer teachers from nine countries, to understand how these educators plan to adapt their teaching methods given to the increasing popularity of AI code-generation tools. As revealed by the interviews, nearly all educators tended to disable such tools in the short term due to the academic integrity concerns brought by AI. From a long-term perspective, some teachers remained opposed to AI code-generation tools, while others recognized these tools, applied them in the classroom, and developed a series of strategies for the application of these tools in the process of programming teaching, including "designing assignments where AI cannot provide direct assistance, reverting to paper exams, setting oral or video assessments, and establishing process-based evaluations."

In summary, regarding the potential implications and strategies to address AI code-generation tools, speculative studies are commonly conducted to explore future societal and educational developments, with the corresponding strategies proposed for talent development and educational reform. This is accompanied by an emerging trend that thought experiment is conducted through speculative science fiction. For adaptation to the evolution of technological trends, the current human practices are guided by envisioning the societal impacts of technological developments and applications (Gu & Cai, 2021). This enriches the research on educational technology. At present, the impact of AI on the programming labor market and educational instruction has been perceived by researchers. Usually taking a proactive and adaptive approach, they seek to accommodate technological influences through a positive adjustment to the training objectives and instructional focus of programming education. However, no consensus has been reached on the practical and feasible methods of educational reform. Additionally, there is a crucial insight gained from the study of Jayagopal et al. (2022), that

is, educational reform isn't straightforward. The reform to education may not be effectively carried out by simply introducing AI code-generation tools into teaching practices. Thus, it is necessary to find effective solutions to empowering educational instruction with AI. It remains challenging but promising to conduct research on this in the future.

Exploration of Behavioral Patterns in Technological Application. Based on a series of studies on the efficiency, performance, current applications, potential opportunities, and risks associated with automated code-generation tools, empirical study has been conducted by foreign researchers to explore the programming behavior of programmers paired with AI. Currently, there are 5 papers focusing on this aspect.

Barke et al. (2023) adopted the Grounded Theory to observe the programming processes of 20 participants. During the observation, the participants applied Github Copilot to complete the same programming task in four different programming languages. It revealed two modes of interaction between the participants and the AI programming assistant. Under the "acceleration" mode, the programmers knew what to do next and used Copilot to achieve their goals more rapidly. Under the "exploration" mode, the programmers unaware of their next steps used Copilot to explore their options. Based on the fundamental theoretical analysis of "how programmers interact with Copilot," the researchers suggested how to enhance the usability of future AI programming assistants.

Additionally, Mozannar et al. (2022) conducted an empirical study on 21 programmers. To begin with, they exercised their developed tagging tool (CUPS) to tag the coding behaviors of programmers manually. Then, over 3000 tagged instances were analyzed and programming behavior was visualized using timelines and state machines to analyze the interaction behaviors between programmers and the code generation system. It was found that programmers spent an average of 34.4% of their time reviewing and editing the suggestions of Copilot when it was provided to complete programming tasks. More than 50% of their time was spent on Copilot-related behaviors. Programmers might rush into accepting Copilot's suggestions when it is proposed but then spend time checking them repeatedly. With new insights into the distribution, patterns, and time costs of different behaviors among programmers, the study carried implications for future interventions in paired programming with AI to enhance the human-machine interaction in AI-assisted programming.

Furthermore, Alves and Cipriano (2023) proposed several collaboration models for programming alongside AI through speculative research, under the assumption that the collaboration between humans and AI is more effective than the use of AI alone. These models include the guiding model (human programmers explicitly instruct AI about task goals, and AI provides solutions), the sketching model (programmers outline the structure of the program, and AI fills in the blanks), and the reverse control model (AI asks programmers what they intend to do until the task goals and constraints are fully understood). Additionally, Pedro suggested that a more efficient environment for AI-enhanced programming should be created by universities for future programmers and

the corresponding ethical guidelines should be developed by them for applying AI technology.

In summary, despite the small number of studies exploring behavioral patterns in technological application, they are significant to educational researchers as a reference for effective decision-making in educational reforms and practices. Nevertheless, there are few empirical studies conducted to observe and collect the behavioral data of programmers paired with AI in coding, with the "black box" of the programming process visualized through coding analysis. They lay a solid foundation for educators to purposefully guide and improve teaching practices, which ensures educational effectiveness.

Ethical and Safety Issues. There are 12 publications focusing on the ethical and security issues related to AI code generation.

Bull and Kharrufa (2023) conducted exploratory interviews with professionals in programming to understand the problems and challenges presented by the rapid development of AI-assisted programming technologies. On this basis, the potential ethical and security issues with AI programming applications were summarized into three aspects: copyright issues, code vulnerabilities, and trustworthy yet incorrect content. Through a survey of teachers and students about the potential threats of applying LLM in programming education, Rahman and Watanobe (2023) summarized them into three aspects: academic misconduct, moral influences and potential biases, and the excessive reliance of learners. Azuaje Pirela (2023) adopted the framework of the World Intellectual Property Organization regarding intellectual property and cutting-edge technologies to identify three major challenges encountered by the current intellectual property agenda: authorship and ownership of algorithmic creation, algorithm transparency, and the metaverse. During a discussion about the copyright interests in open datasets used for training large language models (LLMs), Choksi and Goedicke (2023) outlined the obstacles created by generative programming assistants for copyrights. Besides, a practical roadmap of copyright analysis was drawn for developers, software law experts, and general users to consider in the context of intelligent LLM-driven writing tools. Through an exploratory case study focusing on popular PyPi packages, Go et al. (2023) analyzed their source code using regular expressions (similar to GitHub's inbound search engine) to explore the detectability of these simple vulnerabilities and the exploitability of "code search" technologies. According to the study, the lower versions of packages are more vulnerable to attacks, and it is relatively easy to achieve the malicious application of this engine. Xiao et al. (2022) addressed the potential academic plagiarism issues related to code generation tools like GitHub Copilot, emphasizing the urgent need to promote academic fairness in the era of generative AI by developing the tools applicable to combat these new models, particularly those available ones. In this paper, the ethical and security issues that might arise from the application of AI-assisted programming tools were summarized and categorized (Table 6).

Table 6. Ethical and Security Issues.

Ethics and safety issues	Specific connotations
Academic integrity	It provides a low-risk/high-reward pathway for students focused on short-term outcomes, seeking assistance from AI-assisted programming tools to complete academic tasks rather than developing a profound understanding of the content
Copyright	GitHub copilot uses OpenAI's codex, which is trained on open-source software projects, raising copyright concerns
Sustainability	AI-assisted programming tools may not be as efficient in terms of computational power and network traffic as more mature web services, posing sustainability issues that users should be aware of (Becker et al., 2023)
Fake information	There might be code that looks convincing but is incorrect or code that is functionally correct but may not achieve the expected or required results
Error information	The generated code may contain bugs
Bias	Codex can generate code that reflects stereotypes based on gender, race, emotions, class, naming structures, and other characteristics

The existing research on relevant ethical and safety issues to AI code-generation tools relies mainly on speculative approaches. However, few studies are revealing the problems and risks in the form of case studies through interview methods, experimental approaches, or by reproducing technical implementation phases of AI code generation tools. The research on ethical and safety issues should not be limited to the theoretical level. In future studies, it is essential to rely on the prior research related to ethical issues in AI education, deepen and enrich their content, further guide practice, and link the theoretical and practical aspects cyclically.

General Trends in the Field of Research

Two Paradigms of Research Methodology. The research on AI code-generation tools overall presents two paradigms in terms of research methods.

The existing research exhibits two paradigms in terms of research methods. Firstly, concerning ethical and security issues and the potential impact of technological applications, there is an emphasis on speculative exploratory research. This involves preliminary exploration of the research questions, analyzing the proposed problems to form a basic understanding of the research object, and proposing future research questions and hypotheses. Secondly, regarding issues related to technological applications, there is a notable practical characteristic. A substantial amount of empirical research focuses on the performance evaluation of AI-assisted programming, validating the quality of code generated by AI code-generation tools, and assessing their impact on programmers' programming efficiency. Few studies attempt to analyze the behavioral

patterns of programmers paired with AI for coding. These studies are conducted through qualitative research methods, with limited sample sizes and insufficient representativeness, posing difficulties and risks for theory generalization.

Focus on Risks Brought About by AI. The current research generally focuses on the educational application risks brought about by AI code-generation tools.

From a technical standpoint, issues related to data annotation, comprehension, usage, and security play a crucial role in determining the credibility of AI applications. The quality of datasets and the models used in natural language processing are pivotal factors in determining the quality of AI-generated code. In practical usage, AI-generated content might contain incorrect or fabricated information, struggle to provide suitable answers for everyday knowledge-based questions efficiently, and could have temporal limitations due to the time range of training data. These factors are considered critical limitations of AI programming tools, referred to as "AI Hallucinations" (Alkaissi & McFarlane, 2023). Concurrently, student plagiarism is a prevalent concern within educational institutions. Existing studies indicate that students who use ChatGPT to complete assignments are more likely to engage in plagiarism compared to those who do not use such tools (Bašić et al., 2023). The existence of code-generation tools like ChatGPT might encourage students to see them as shortcuts, potentially fostering an environment of academic dishonesty and creating an unfair learning competition (Cotton et al., 2024). These issues are widely addressed in current research, and some researchers view them as an opportunity to explore ways to mitigate the risks associated with AI code-generation tools, which are significantly valuable for the future of education and teaching.

Research Needs to Be Deepened. The educational impact and application value generated by AI code-generation tools have not yet received sufficient attention.

Overall, there is a limited amount of research on the impact of AI code-generation tools on programming education. Existing studies generally lack descriptive and explanatory research on the issues and are not deep enough. To a certain extent, within the current trend of technological change, academia's research on programming education lags behind its technical development. A few researchers have recognized the urgent need for a shift in talent development in the context of AI-empowered programming. However, their research is at a lower level, with one-sided research methods, insufficient research conclusions, and a lack of practical guidance significance. Existing research still needs further deepening.

Implications for Future Research

As technology advances and becomes more ubiquitous, AI code-generation tools are poised to transcend their role as external tools in educational settings. Instead, they will progressively infiltrate all aspects of educational teaching and learning, becoming integral to educational subjects, activities, content, and environments. This integration

will set off a chain reaction within the educational ecosystem (Lu & Wang, 2019), signifying an inevitable trend of technology permeating educational systems. Confronted with the impact of such tools on the educational landscape, "educational leadership" emerges as a crucial response strategy. A systematic analysis of existing literature provides enlightening insights for future academic research about the practice of "AI-human pairing" in programming.

Research Approach: Clarify Technical Flaws and Ethical Risks. While acknowledging the value of AI code generation tools, it's equally important to delineate their technical limitations and ethical risks.

Literature research indicates that GAI tools possess potential value in empowering teaching and learning. On one hand, these tools aid educators in creating teaching materials, thereby enhancing work efficiency (Gabajiwala et al., 2022). On the other hand, they serve as personalized "learning companions" for students, assisting them in better learning based on their circumstances (Ali et al., 2023; Ji et al., 2022). Similarly, the process of interacting with AI requires students to guide the AI in generating accurate content through natural language and critically evaluate the AI's content output. This process contributes to the development of students' higher-order thinking skills such as computational thinking and critical thinking (Abdelghani et al., 2022). However, AI code-generation tools also carry certain application risks. The existence of "AI hallucinations" may lead learners to develop cognitive biases. Misusing AI can result in academic ethical and moral risks, leading to "ineffective learning" during the teaching process (Ali et al., 2023). AI empowering education is an inevitable trend in technological development. In the integration of AI-assisted programming tools with education, new ethical risks will emerge in educational technology, teacher-student relationships, and the learning environment. Therefore, future research should further enrich ethical reflections on the educational applications of AI-assisted programming tools to prevent potential issues.

Research Content: Building Adaptive Teaching Ecologies and Educational Models. An adaptive teaching environment and an educational model should be developed, and comprehensive empirical research should be conducted to explore the enhancement mechanism of human-machine paired programming.

Before the widespread discussion about whether artificial intelligence can replace existing professions was triggered by the release of ChatGPT 3.5, tech companies were already pioneering the integration of virtual assistants based on generative artificial intelligence into their ecosystem. The McKinsey Global Institute (2017), in a report released 6 years ago, made predictions that with technological advancements, by the year 2030, a conservative estimate suggests that around 15% of the global workforce (approximately 400 million people) could experience job displacement due to artificial intelligence. The rapid adoption of GAI technology in the industrial sector might instantaneously disrupt the labor market landscape. AI is poised to become a significant driver of productivity innovation and labor revolution, with efficient pairing with AI

becoming mainstream across various industries. In the realm of computer engineering and software development, there is a high likelihood of AI replacing tasks involving lower-level code writing. Failing to adapt to technological advancements might place individuals in a disadvantaged position during this technological revolution.

In the field of computer engineering and software development, AI, with its vast data support, powerful learning capabilities, and integration of knowledge from the entire Internet, can directly replace many tasks that traditionally require human labor. Current research indicates that when faced with simple programming problems, AI is more efficient compared to humans. However, simultaneously, when dealing with complex programming issues, machines currently lag behind humans in terms of business logic analysis and processing capabilities. As shown in Figure 3, breaking down a complete program file reveals a multitude of computational expressions, which are relatively simple to implement and form the foundation of the entire program, located at the bottom of the pyramid. In contrast, higher levels consist of functions and modules designed according to algorithms, ultimately achieving complete functionality. To better integrate human and machine intelligence, pairing programmers with AI is essential. In future human-AI pair programming, humans will take the lead, designing the logical processing tasks, while machines handle the low-level code writing. It is crucial to recognize the dual nature of machine capabilities, leveraging human intelligence to maximize the value of technology.

From an ecological perspective, educational objectives and governance are undergoing varying degrees of reshaping due to technological changes. The educational ecosystem is experiencing a disruptive trend under the influence of technology, expanding the connotations of various elements within the system and reshaping the functions and forms of the educational system (Gu et al., 2017). As the core link for supplying talent to social production and the main ground for cultivating high-level talent, universities' success in talent cultivation is not solely measured by the accumulation of knowledge. Instead, the focus should be on the development of higher-order thinking abilities, considering the quality of talent required for the future computer industry, and developing adaptive teaching ecosystems and educational models for the era of generative artificial intelligence.

In this context, academic research should be conducted extensively at both macro and micro levels (as shown in Figure 4).

At the macro level, research should concentrate on the programming education ecosystem reshaped by technological changes. This involves analyzing and reconstructing the new aspects of various components within the higher education programming education ecosystem under the impact of new technologies such as generative artificial intelligence. It further entails clarifying programming education objectives, delineating the educational needs of stakeholders, specifying educational service recipients and content, and establishing strategies and paths for educational governance.

At the micro level, reforms and practical pathways for university programming teaching should be proposed from the overall perspectives of teachers, students, and

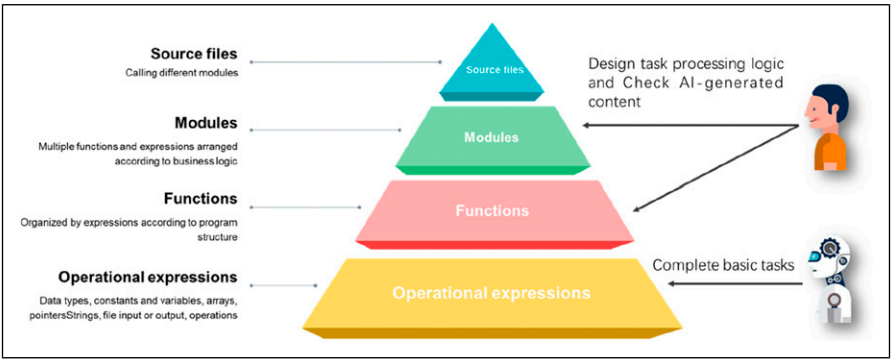

Figure 3. Future human-machine pairing programming model.

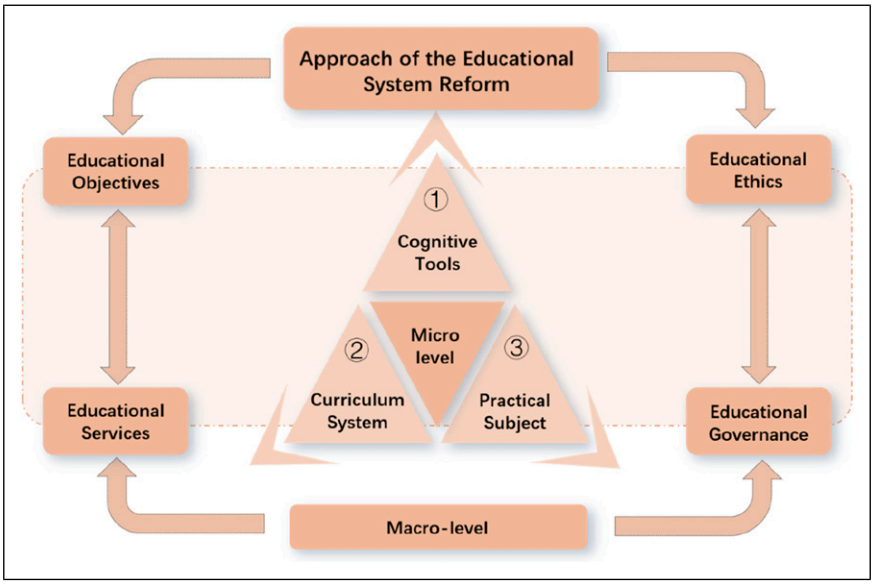

Figure 4. Focus of research.

courses. This includes in-depth research on the potential and risks of using auxiliary programming tools in teaching, developing and designing core elements of programming course teaching around educational objectives, utilizing learning analytics technology to explore the "black box" of students' learning processes in AI-paired programming, and constructing a competency model for teachers within the new educational ecosystem. Based on these approaches, comprehensive teaching practices

should be implemented to achieve a beneficial cycle of "theory-driven practice, refining theory," reshaping the future educational ecosystem propelled by technological changes.

Research Methods: Enriching the Application of Research Methods. Further enrichment of research methods should combine speculative research with empirical research, integrating traditional methods with emerging technologies.

The current research on emerging technology empowering education mainly relies on speculative research methods. In the future exploration of the theory and practice of "human-machine pairing" in programming, it is imperative to fully engage in educational practices, combining speculative and empirical research, based on the perspective of complex systems science. This approach leverages theoretical research for strategic guidance, integrating educational research findings with practical applications, thus creating a closed loop of "theory-driven practice, practice sublimated into theory."

Furthermore, in the process of collaborating with AI to accomplish tasks, humans need to continually engage in autonomous questioning, analysis, and critique of AI-generated content. Actively participating in problem-solving-oriented knowledge construction by issuing repeated commands and probing deeper questions aids in better understanding current issues and AI's content output. This intricate process, full of possibilities, isn't effectively visualized through traditional survey research methods. Some emerging research methods facilitate understanding the essence of technological applications. Learning Analytics (LA) is one such emerging technology in the educational technology field. Scholars describe learning analytics as "an emerging field that uses sophisticated analytical tools to improve learning and education" (Chen et al., 2022). Utilizing learning analytics technology enables the visualization of the learning process, providing insights into problems and proposing strategies to optimize the learning effectiveness for learners.

Just as various technologies like sequence analysis, cognitive network analysis, and multimodal analysis have widely been employed in analyzing learners' learning processes, future research into AI-assisted programming processes requires comprehensive studies to unravel the "black box" of programmers' coding processes. It should delve deeper into understanding the mechanisms for enhanced learning or productivity while collaborating with machines. Embracing technology while harnessing human subjectivity allows for steering technology to create conditions that enhance human work efficiency.

Some scholars have utilized speculative fiction as a vehicle to conduct thought experiments, envisioning the future of artificial intelligence and its educational implications (Gu & Cai, 2021). These research methods possess pioneering, leading, and forward-thinking aspects, offering crucial value in foreseeing technological application scenarios, and challenges, and premeditating strategies for handling them. In the realm of human-machine pairing programming research, speculative fiction offers a forward-looking perspective to envisage future modes and processes of human-machine

collaboration. This foresight can drive educational systems to adapt promptly and reform to address the new socio-technical landscape brought about by AI.

Conclusion and Discussion

This study employed a thematic analysis method based on qualitative coding to examine the current status of educational applications and related research on AI code-generation tools from 2022 to the present. The results indicate that AI code-generation tools have not received sufficient attention in the field of education. Only a few studies have recognized the application value of such tools in human-machine paired programming. Existing research focuses on five aspects: fundamental technology and tool introductions, practical applications of AI-assisted programming tools, potential impacts and coping strategies in educational applications, exploration of behavioral patterns in technology applications, and ethical and security issues.

Based on the research findings, we propose future research recommendations for AI-assisted programming in the education domain from both content and methodological perspectives. Considering the perspective of "co-learning" between humans and artificial intelligence, future research should focus on three aspects: (1) Acknowledging the application value of AI code generation tools while clarifying their technical limitations and ethical risks; (2) Developing adaptive teaching ecosystems and nurturing models, conducting comprehensive empirical research to explore the efficiency mechanisms of human-machine paired programming; (3) Further enriching the application of research methods by combining speculative and empirical studies, integrating traditional methods with emerging technologies.

The current research on the educational applications of AI code-generation tools is limited and needs further exploration. As technology matures, the inevitable penetration of these tools into educational systems requires a proactive approach to educational research. To address the personalized learning difficulties commonly faced by novice programmers and align with the future talent needs of the labor market, collaborative programming is widely practiced in higher education programming education. The emergence of AI code generation tools has introduced a new form of "human-machine pair programming" for collaborative coding. The focus should be on what kind of AI is applied to programming education and how to effectively integrate AI into programming education.

In the future, higher education should take a dual approach, focusing on both "human" and "machine," to create a new paradigm for programming education that integrates the trinity of "teacher-student-machine."

For "humans," educators are always at the core of the race with technological development. In order to cope with the upcoming technological singularity, education should take the lead, strategically planning the future talent structure. With the increasing demand for "human-machine pairing" in the future job market, higher education programming education should create more opportunities for students to engage in "human-machine collaboration" programming practices. It is crucial to

maximize the integration of human and machine intelligence and build teaching practices upon this foundation. Simultaneously, universities should place greater emphasis on cultivating students' computational thinking. Diverging from a reliance solely on memorizing programming syntax and rules, the emphasis on developing students' computational thinking underscores the understanding of problem essence, the ability to seek solutions, and the capacity to transform these solutions into effective algorithms. Under this philosophy, students are encouraged to adopt a problem-centric approach, formulating solutions through analyzing the structure and requirements of the problem. At a macro level, comprehension of the problem's premise precedes the pursuit of micro-level syntax details. The ability to "read" and "evaluate" code should be included in the scope of educational assessments. Meanwhile, higher education institutions should expedite the development of globally oriented and strategically flexible higher education curriculum plans and talent cultivation programs (Jiao et al., 2023) to adapt to the impact of technology on the future labor market. Through the application and enhancement of human wisdom and "transformative power," we can make this technological storm generate educational effectiveness and play a role in nurturing individuals.

For "machines," not all human-machine interactions in programming have value. Open interaction between students and AI may lead to students forming a dependence on technology, hindering the development of their innovative thinking and problem-solving abilities. How to avoid the "problem-answer" interaction between learners and AI in the programming process and establish conditions for collaborative problem-solving processes is a pressing issue. This is not only a challenge that emerging code generation tools urgently need to overcome in the future application in the field of education but also a key factor for ensuring that new technologies truly empower education, support educational entities, achieve precise teaching, and effectively serve the entire education system. After minimizing the negative impact of machines on learning as much as possible, universities should gradually introduce technology into the classroom, fully leveraging the functions of "teaching" and "research," summarizing experiences in practice.

In conclusion, faced with the emerging technology of AI code generation, we should neither blindly extol the power of "giant machines" in AI programming (Mumford, 1971), nor confine it within the "forbidden zone" solely based on the current drawbacks of AI programming technology. We need to dialectically assess and rationally apply the strengths and weaknesses of technology, gradually introducing technology into classroom teaching and exploring more possibilities for technology to empower education.

Author Contributions

JL: Conceptualization, Methodology, Investigation, Writing—original draft, Project administration, Funding acquisition, Supervision; SL: Methodology, Investigation, Writing—original draft.

Declaration of Conflicting Interests

The author(s) declared no potential conflicts of interest with respect to the research, authorship, and/or publication of this article.

Funding

The author(s) disclosed receipt of the following financial support for the research, authorship, and/or publication of this article: This work was supported by 2023 Jiangsu Province Education Science Planning Project [B/2023/01/193].

Ethical Statement

Ethical Approval

Research ethics approval was obtained by the Institutional Review Board of Soochow University.

ORCID iD

Siran Li https://orcid.org/0000-0002-1511-8189

Data Availability Statement

The datasets used and/or analyzed during the current study are available from the corresponding author on reasonable request.

References

Abdelghani, R., Oudeyer, P.-Y., Law, E., de Vulpillières, C., & Sauzéon, H. (2022). Conversational agents for fostering curiosity-driven learning in children. *International Journal of Human-Computer Studies*, *167*(11), 102887. https://doi.org/10.1016/j.ijhcs.2022.102887

Ali, H., Qadir, J., & Shah, Z. (2023). *ChatGPT and Large Language Models (LLMs) in healthcare: Opportunities and risks*. TechRxiv. https://doi.org/10.36227/techrxiv.22579852.v2

Alkaissi, H., & McFarlane, S. I. (2023). Artificial hallucinations in ChatGPT: Implications in scientific writing. *Cureus*, *15*(2), Article e35179. https://doi.org/10.7759/cureus.35179

Alves, P., & Cipriano, B. P. (2023). *The centaur programmer–How Kasparov's Advanced Chess spans over to the software development of the future*. arXiv. preprint arXiv:2304.11172.

Azuaje Pirela, M. (2023). Three major challenges for the intellectual property agenda in the face of artificial intelligence and other frontier technologies. *Available at SSRN 4378436*. https://doi.org/10.2139/ssrn.4378436

Barke, S., James, M. B., & Polikarpova, N. (2023). Grounded copilot: How programmers interact with code-generating models. *Proceedings of the ACM on Programming Languages*, *7*(OOPSLA1), 85–111. https://doi.org/10.1145/3586030

Bašić, Ž., Banovac, A., Kružić, I., & Jerković, I. (2023). ChatGPT-3.5 as writing assistance in students' essays. *Humanities and Social Sciences Communications*, *10*(1), 1–5. https://doi.org/10.1057/s41599-023-02269-7

Beck, L., & Chizhik, A. (2013). Cooperative learning instructional methods for CS1: Design, implementation, and evaluation. *ACM Transactions on Computing Education (TOCE)*, *13*(3), 1–21. https://doi.org/10.1145/2499947.2492686

Becker, B. A., Denny, P., Finnie-Ansley, J., Luxton-Reilly, A., Prather, J., & Santos, E. A. (March, 2023 15-18). Programming is hard-or at least it used to be: Educational opportunities and challenges of ai code generation [Conference session]. In Proceedings of the 54th ACM Technical Symposium on Computer Science Education, Toronto, ON, Canada, (Vol. 1). https://doi.org/10.1145/3545945.3569759

Bennedsen, J., & Caspersen, M. E. (2007). Failure rates in introductory programming. *ACM SIGCSE Bulletin*, *39*(2), 32–36. https://doi.org/10.1145/1272848.1272879

Bird, C., Ford, D., Zimmermann, T., Forsgren, N., Kalliamvakou, E., Lowdermilk, T., & Gazit, I. (2022). Taking Flight with Copilot: Early insights and opportunities of AI-powered pair-programming tools. *Queue*, *20*(6), 35–57. https://doi.org/10.1145/3582083

Bull, C., & Kharrufa, A. (2023). Generative AI assistants in software development education: A vision for integrating generative AI into educational practice, not instinctively defending against it. *IEEE Software*, *41*(2), 52–59. https://doi.org/10.1109/ms.2023.3300574

Chen, M. X., Zhou, L., & Zhao, J. Y. (2022). The link between learning design and learning analytics: Current situation, challenges and implementation paths. *Open Education Research*, *28*(06), 27–36 (in Chinese). https://doi.org/10.13966/j.cnki.kfjyyj.2022.06.004

Chittum, J. R., Jones, B. D., Akalin, S., & Schram, Á. B. (2017). The effects of an afterschool STEM program on students' motivation and engagement. *International Journal of STEM Education*, *4*(1), 11–16. https://doi.org/10.1186/s40594-017-0065-4

Choksi, M. Z., & Goedicke, D. (2023). *Whose text is it anyway? Exploring BigCode, intellectual property, and ethics*. arXiv. preprint arXiv:2304.02839.

Cotton, D. R., Cotton, P. A., & Shipway, J. R. (2024). Chatting and cheating: Ensuring academic integrity in the era of ChatGPT. *Innovations in Education and Teaching International*, *61*(2), 228–239. https://doi.org/10.1080/14703297.2023.2190148

Dakhel, A. M., Majdinasab, V., Nikanjam, A., Khomh, F., Desmarais, M. C., & Jiang, Z. M. J. (2023). Github copilot ai pair programmer: Asset or liability? Journal of Systems and Software, *203*(9), Article 111734. https://doi.org/10.1016/j.jss.2023.111734

Denny, P., Kumar, V., & Giacaman, N. (2023, March 15-18). Conversing with copilot: Exploring prompt engineering for solving cs1 problems using natural language [Conference session]. The 54th ACM Technical Symposium on Computer Science Education, Toronto, Canada. https://doi.org/10.1145/3545945.3569823

Denny, P., Sarsa, S., Hellas, A., & Leinonen, J. (2022). *Robosourcing educational resources–leveraging Large Language Models for learnersourcing*. arXiv. preprint arXiv:2211.04715.

Eysenbach, G. (2023). The role of ChatGPT, generative language models, and artificial intelligence in medical education: A conversation with ChatGPT and a call for papers. *JMIR Medical Education*, *9*(1), Article e46885. https://doi.org/10.2196/46885

Finnie-Ansley, J., Denny, P., Becker, B. A., Luxton-Reilly, A., & Prather, J. (2022, February 14-18). The robots are coming: Exploring the implications of openai codex on introductory programming[conference session]. 24th Australasian Computing Education Conference, Virtual Event, Australia. https://doi.org/10.1145/3511861.3511863

Finnie-Ansley, J., Denny, P., Luxton-Reilly, A., Santos, E. A., Prather, J., & Becker, B. A. (2023, January 30 - February 2). My AI wants to know if this will Be on the exam: Testing OpenAI's Codex on CS2 programming Exercises [conference session]. 25th Australasian Computing Education Conference, Melbourne, Australia. https://doi.org/10.1145/3576123.3576134

Gabajiwala, E., Mehta, P., Singh, R., & Koshy, R. (2022). Quiz Maker: Automatic Quiz Generation from Text Using NLP. In P.K. Singh, S.T. Wierzchoń, J.K. Chhabra, & S. Tanwar (Eds.), *Futuristic Trends in Networks and Computing Technologies. Lecture Notes in Electrical Engineering* (*936*, pp. 523–533). Singapore: Springer. https://doi.org/10.1007/978-981-19-5037-7_37

Go, K. R., Soundarapandian, S., Mitra, A., Vidoni, M., & Ferreyra, N. E. D. (2023). Simple stupid insecure practices and GitHub's code search: A looming threat? *Journal of Systems and Software*, *202*(8), Article 111698. https://doi.org/10.1016/j.jss.2023.111698

Gu, X. Q., & Cai, H. Y. (2021). Predicting the future of artificial intelligence and its impact on education: A thought experiment based on social science fiction. *Educational Research*, *42*(05), 137–147 (in Chinese). https://kns.cnki.net/kcms2/article/abstract?v=aHgEko1xHjioUYqahxpiDVdEkvUgT-mOxD4O1vlZFOlokgIYaEAT37Puen6F9QoiEMulz17eBHatLhCHlp0zWhXaWRzq3jbppMeUeYfk1TX0FNlIcTnwWG55md_3tq-8W7OnbhkIA1w=&uniplatform=NZKPT&flag=copy

Gu, X. Q., Wang, C. L., & Wang, F. (2017). Historical mission and roles of ICT in educational reform in the past 20 years. *E-Education Research*, *38*(06), 9–19. (in Chinese). https://doi.org/10.13811/j.cnki.eer.2017.06.002

Häkkinen, P., Arvaja, M., Hämäläinen, R., & Pöysä, J. (2010). *Scripting computer-supported collaborative learning: A review of SCORE studies. E-Collaborative knowledge construction: Learning from computer-supported and virtual environments* (pp. 180–194). IGI Global. https://doi.org/10.4018/978-1-61520-729-9.ch010

Hindle, A., Barr, E. T., Gabel, M., Su, Z., & Devanbu, P. (2016). On the naturalness of software. *Communications of the ACM*, *59*(5), 122–131. https://doi.org/10.1145/2902362

Jayagopal, D. (2022). *Study of program synthesizers and novice programmers*. University of California. [Unpublished master dissertation].

Ji, H., Han, I., & Ko, Y. (2022). A systematic review of conversational AI in language education: Focusing on the collaboration with human teachers. *Journal of Research on Technology in Education*, *55*(1), 48–63. https://doi.org/10.1080/15391523.2022.2142873

Jiao, J. L., Chen, L., & Wu, W. W. (2023). Educational issues triggered by ChatGPT: Possible impacts and counter measures. *Chinese Journal of ICT in Education*, *29*(03), 19–32. (in Chinese). https://doi.org/10.3969/j.issn.1673-8454.2023.03.003

Krings, K., Bohn, N. S., Hille, N. A. L., & Ludwig, T. (2023, April 23-28). "What if everyone is able to program?"–exploring the role of software development in science fiction [conference session]. CHI Conference on Human Factors in Computing Systems 2023, Hamburg, Germany. https://doi.org/10.1145/3544548.3581436

Lau, S., & Guo, P. (2023, August 7-11). From "Ban it till we understand it" to "Resistance is futile": How university programming instructors plan to adapt as more students use AI code generation and explanation tools such as ChatGPT and GitHub copilot [conference session].

2023 ACM Conference on International Computing Education Research-Volume 1, Chicago USA. https://doi.org/10.1145/3568813.3600138

Liberati, A., Altman, D. G., Tetzlaff, J., Mulrow, C., Gøtzsche, P. C., Ioannidis, J. P., Clarke, M., Devereaux, P. J., Kleijnen, J., Moher, D., & Moher, D. (2009). The PRISMA statement for reporting systematic reviews and meta-analyses of studies that evaluate health care interventions: Explanation and elaboration. *Annals of Internal Medicine*, *151*(4), W65. W94, https://doi.org/10.7326/0003-4819-151-4-200908180-00136

Lu, X. Z., & Wang, S. L. (2019). Historical thinking and future development of educational informatization in China: Based on the relationship between educational informatization and educational modernization. *Jiangsu Higher Education*, *34*(12), 1–8 (in Chinese). https://doi.org/10.13236/j.cnki.jshe.2019.12.001

MacNeil, S., Tran, A., Mogil, D., Bernstein, S., Ross, E., & Huang, Z. (2022, August 7-11). Generating diverse code explanations using the gpt-3 large language model [Conference session]. ICER 2022: ACM Conference on International Computing Education Research, Lugano and Virtual Event, Switzerland. https://doi.org/10.1145/3501709.3544280

Markel, J. M., & Guo, P. J. (2021, March). Inside the mind of a cs undergraduate ta: A firsthand account of undergraduate peer tutoring in computer labs [conference session]. The 52nd ACM Technical Symposium on Computer Science Education, Virtual Event, USA. https://doi.org/10.1145/3408877.3432533

Mastropaolo, A., Pascarella, L., Guglielmi, E., Ciniselli, M., Scalabrino, S., Oliveto, R., & Bavota, G. (2023). *On the robustness of code generation techniques: An empirical study on github copilot*. arXiv. preprint arXiv:2302.00438.

McKinsey Global Institute. (2017, November 28). *Jobs Lost, jobs gained: Workforce Transitions in a time of automation*. https://www.mckinsey.com/featured-insights/future-of-work/jobs-lost-jobs-gained-what-the-future-of-work-will-mean-for-jobs-skills-and-wages

Medeiros, R. P., Ramalho, G. L., & Falcao, T. P. (2019). A systematic literature review on teaching and learning introductory programming in higher education. *IEEE Transactions on Education*, *62*(2), 77–90. https://doi.org/10.1109/te.2018.2864133

Megahed, F. M., Chen, Y. J., Ferris, J. A., Knoth, S., & Jones-Farmer, L. A. (2023). How generative ai models such as chatgpt can be (mis) used in spc practice, education, and research? An exploratory study. *Quality Engineering*, *36*(2), 287–315. https://doi.org/10.1080/08982112.2023.2206479

Mozannar, H., Bansal, G., Fourney, A., & Horvitz, E. (2022). *Reading between the lines: Modeling user behavior and costs in AI-assisted programming*. arXiv. preprint arXiv: 2210.14306.

Müller, M. M. (2004). Are reviews an alternative to pair programming? *Empirical Software Engineering*, *9*(4), 39–351. https://doi.org/10.1023/b:emse.0000039883.47173.39

Mumford, L. (1971). *Technics and human development: the myth of the machine* (vol. I, pp. 381–410). Harvest Books.

Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). *The impact of ai on developer productivity: Evidence from github copilot*. arXiv. preprint arXiv:2302.06590.

Prather, J., Reeves, B. N., Denny, P., Becker, B. A., Leinonen, J., Luxton-Reilly, A., Powell, G., Finnie-Ansley, J., & Santos, E. A. (2023). *It's weird that it knows what I want: Usability and interactions with copilot for novice programmers*. arXiv. preprint arXiv:2304.02491.

Rahman, M. M., & Watanobe, Y. (2023). ChatGPT for education and research: Opportunities, threats, and strategies. *Applied Sciences*, *13*(9), 5783. https://doi.org/10.20944/preprints202303.0473.v1

Robbes, R., & Lanza, M. (2010). Improving code completion with program history. *Automated Software Engineering*, *17*(2), 181–212. https://doi.org/10.1007/s10515-010-0064-x

Romero, M., Lepage, A., & Lille, B. (2017). Computational thinking development through creative programming in higher education. *International Journal of Educational Technology in Higher Education*, *14*(1), 1–15. https://doi.org/10.1186/s41239-017-0080-z

Rudolph, J., Tan, S., & Tan, S. (2023). ChatGPT: Bullshit spewer or the end of traditional assessments in higher education? *Journal of Applied Learning and Teaching*, *6*(1), 9. https://doi.org/10.37074/jalt.2023.6.1.9

Sarsa, S., Denny, P., Hellas, A., & Leinonen, J. (2022, August 7-11). Automatic generation of programming exercises and code explanations using large language models [Conference session]. ACM conference on International computing education research, Lugano and Virtual Event, Switzerland. https://doi.org/10.1145/3501385.3543957

Schäfer, M., Nadi, S., Eghbali, A., & Tip, F. (2024). An empirical evaluation of using Large Language Models for automated unit test generation. *IEEE Transactions on Software Engineering*, *50*(1), 85–105. https://doi.org/10.1109/tse.2023.3334955

Sfetsos, P., Stamelos, I., Angelis, L., & Deligiannis, I. (2006). Investigating the impact of personality types on communication and collaboration-viability in pair programming–an empirical study. In P. Abrahamsson, M. Marchesi, & G. Succi (Eds.), *Extreme Programming and Agile Processes in Software Engineering* (*4044*, pp. 43–52). Springer. https://doi.org/10.1007/11774129_5.

Simon, W. (2022, November). *14). Writing tests with copilot*. Simon Willison's TILs. https://til.simonwillison.net/gpt3/writing-test-with-copilot.

Sobania, D., Briesch, M., Hanna, C., & Petke, J. (2023). *An analysis of the automatic bug fixing performance of chatgpt*. arXiv. preprint arXiv:2301.08653.

Stahl, G., & Hakkarainen, K. (2021). *Theories of CSCL. International handbook of computer-supported collaborative learning* (pp. 23–43). Springer. https://doi.org/10.4324/9781410601544-21

Vaithilingam, P., Zhang, T., & Glassman, E. L. (2022, April 29 – 2022, May 5). Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models [conference session]. CHI Conference on Human Factors in Computing Systems, Los Angeles, United States. https://doi.org/10.1145/3491101.3519665

Watson, C., & Li, F. W. B. (2014, June 21-25). Failure rates in introductory programming revisited [conference session]. 2014 Conference on Innovation & Technology in Computer Science Education - ITiCSE '14, Uppsala, Sweden. https://doi.org/10.1145/2591708.2591749

Wermelinger, M. (2023). *Using GitHub Copilot to Solve Simple Programming Problems*. Proceedings of the 54th ACM Technical Symposium on Computer Science Education *V.* 1. https://doi.org/10.1145/3545945.3569830

White, M., Vendome, C., Linares-Vásquez, M., & Poshyvanyk, D. (2015, May 16-24). Toward deep learning software repositories [Conference session]. 2015 IEEE/ACM 12th Working Conference on Mining Software Repositories, Florence, Italy. https://doi.org/10.1109/msr.2015.38

Xiao, Y., Chatterjee, S., & Gehringer, E. (2022, November 6-9). A new era of plagiarism the danger of cheating using AI [Conference session]. 20th International Conference on Information Technology Based Higher Education and Training 2022, Antalya, Turkey. https://doi.org/10.1109/ithet56107.2022.10031827

Xu, W., Wu, Y., & Ouyang, F. (2023). Multimodal learning analytics of collaborative patterns during pair programming in higher education. *International Journal of Educational Technology in Higher Education*, *20*(1), 1–20. https://doi.org/10.1186/s41239-022-00377-z

Yang, Z., Chen, S., Gao, C., Li, Z., Li, G., & Lv, R. (2023). *Deep learning based code generation methods: A literature review.* arXiv. preprint arXiv:2303.01056.

Zheng, Q., Xia, X., Zou, X., Dong, Y., Wang, S., Xue, Y., Wang, Z., Shen, L., Wang, A., Li, Y., Su, T., Yang, Z., & Tang, J. (2023). *Codegeex: A pre-trained model for code generation with multilingual evaluations on humaneval-x*. arXiv. preprint arXiv:2303.17568.

Author Biographies

Jiangyue Liu, PhD in Higher Education and M.Sc. in Computer Science and Technology, is an associate professor at Soochow University specializing in relational databases, computer networks, educational resource development, and computer network security. He is actively engaged in research and software development, focusing on teacher education talent cultivation models, educational resource utilization, and management information system development. A recipient of funding from the National Scholarship Council, Liu has undertaken academic visits to the United States and Germany.

Siran Li, B.S. in Educational Technology from Soochow University in 2018, is currently pursuing a master's degree in Educational Technology at the School of Education, Soochow University, Suzhou, Jiangsu, China. Her research interests encompass Education big data, Artificial Intelligence, and Education and Learning Analysis.